Consolidated Security Report

Reported by: Charmaine Mangorima
Date: 9 January 2026

Executive Summary
This report is a consolidated report for all the work submitted from week 1 up to week 4, taking into consideration all the feedback provided and submitting the corrected reports for each week. The report consists of the work from week 1 and week 2 as they are ones that I got feedback that needs corrections. **For week 1**, I exploited the VAmPI API but some of my screenshots for evidence did not flow with the findings, my screenshots did not have highlights(which I addressed in week 4 after the feedback) and some findings did not have descriptions. **For week 2**, I analysed the given JWT but I did not include recommendations for the findings.

Week 1
Assignment: **API Security Assessment Report**
Target Application: **VAmPI API**
Feedback: **Good work. But you don't have a description for some of the findings. Also use highlights in your screenshots to point out important areas. Also, make your screenshots flow with your findings. It helps to put your points across and for reproducing.**
Corrections: **Corrected description for finding 1 and 2, added description to finding 3, increased screenshot sizes.**

Summary

This report analyzes the provided API Swagger specification to identify three critical security flaws, following the OWASP API Security Top 10 methodology. Each finding includes an explanation of the security risk and recommended mitigations.

The identified flaws include:

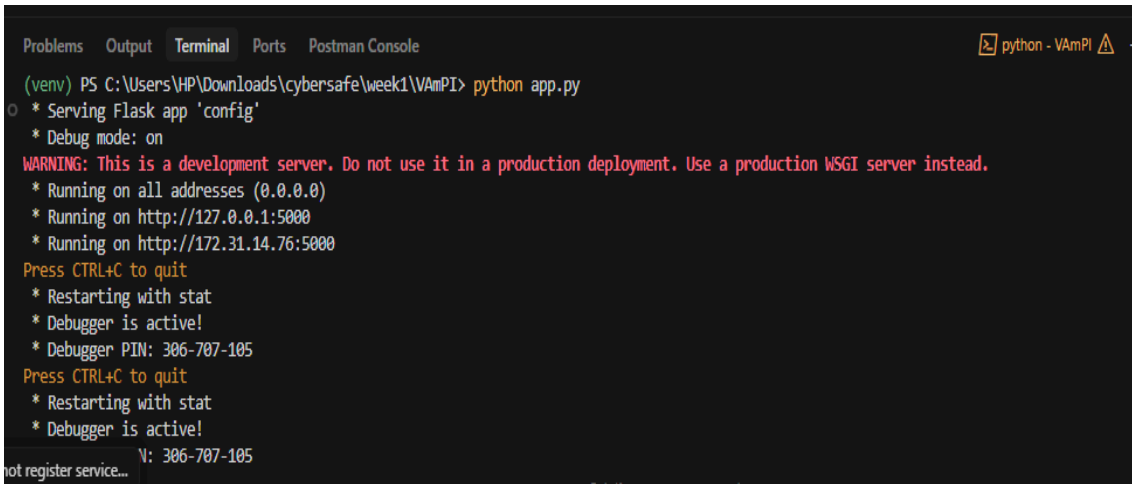
- Broken Object Level Authorization
- Excessive Data Exposure
- Lack of Access Controls on Sensitive Endpoints

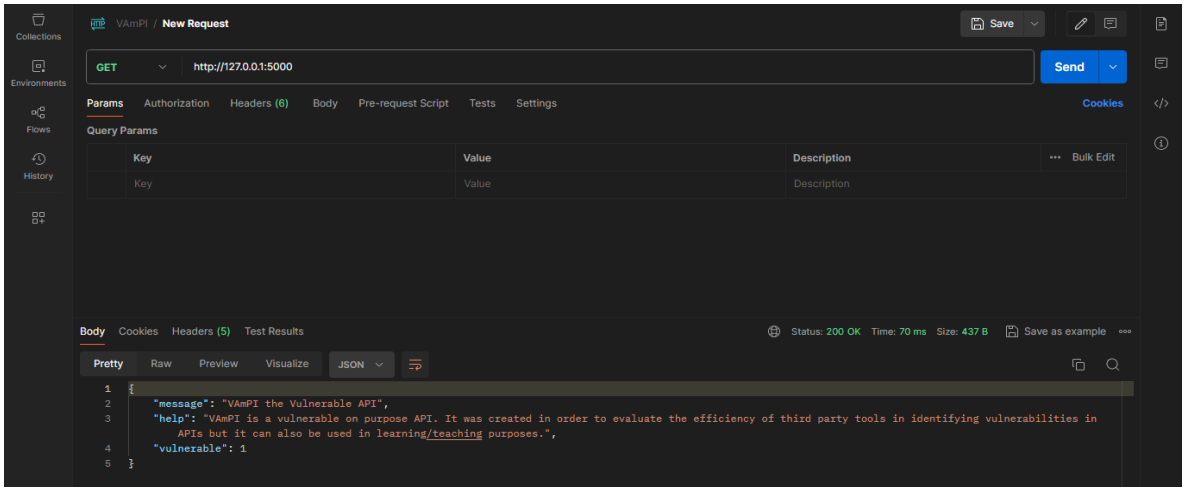
Tools used

- VS Code/Cursor (to start the server)
- Postman (manual API testing)
- Github (https://github.com/erev0s/VAmPI)

Starting the server

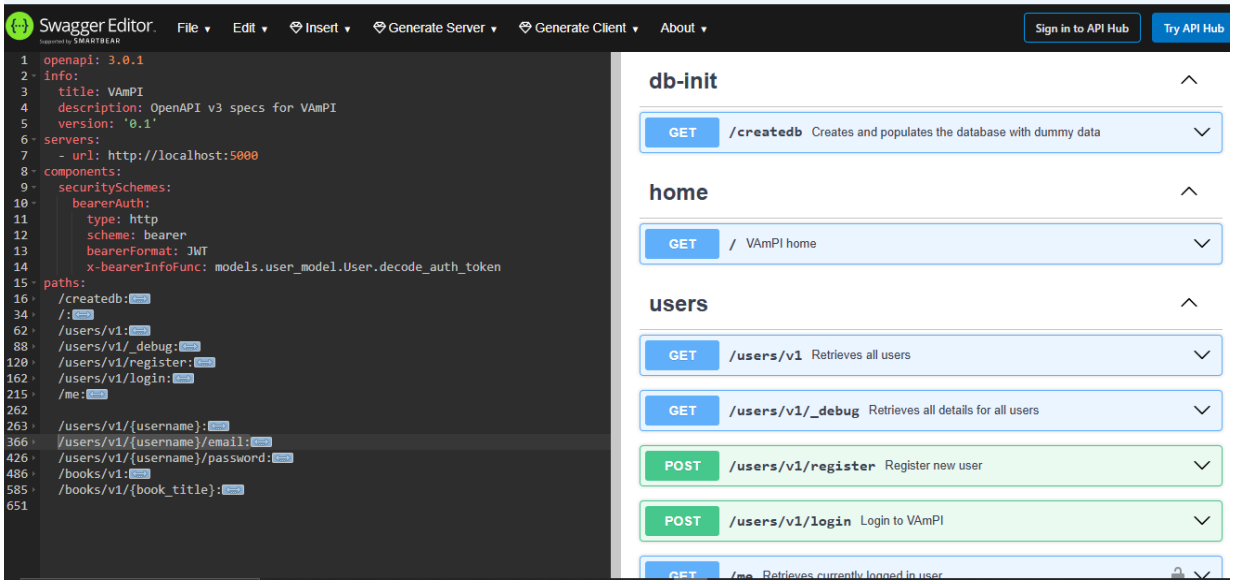
VAmPI is a vulnerable API made with Flask and it includes vulnerabilities from the OWASP top 10 vulnerabilities for APIs. It was created as a vulnerable API to evaluate the efficiency of tools used to detect security issues in APIs. To start this server, I cloned the repository on github, installed all the requirements via **pip install -r requirements.txt** and started the server on my machine and it is running on localhost **http://127.0.0.1:5000** as shown below:





I also used another method by importing the VAmPI swagger specification into Postman.

I made use of Swagger editor to get a clear view of the API endpoints:



Identified Security Flaws

1 Broken Object Level Authorization (BOLA) – OWASP API1

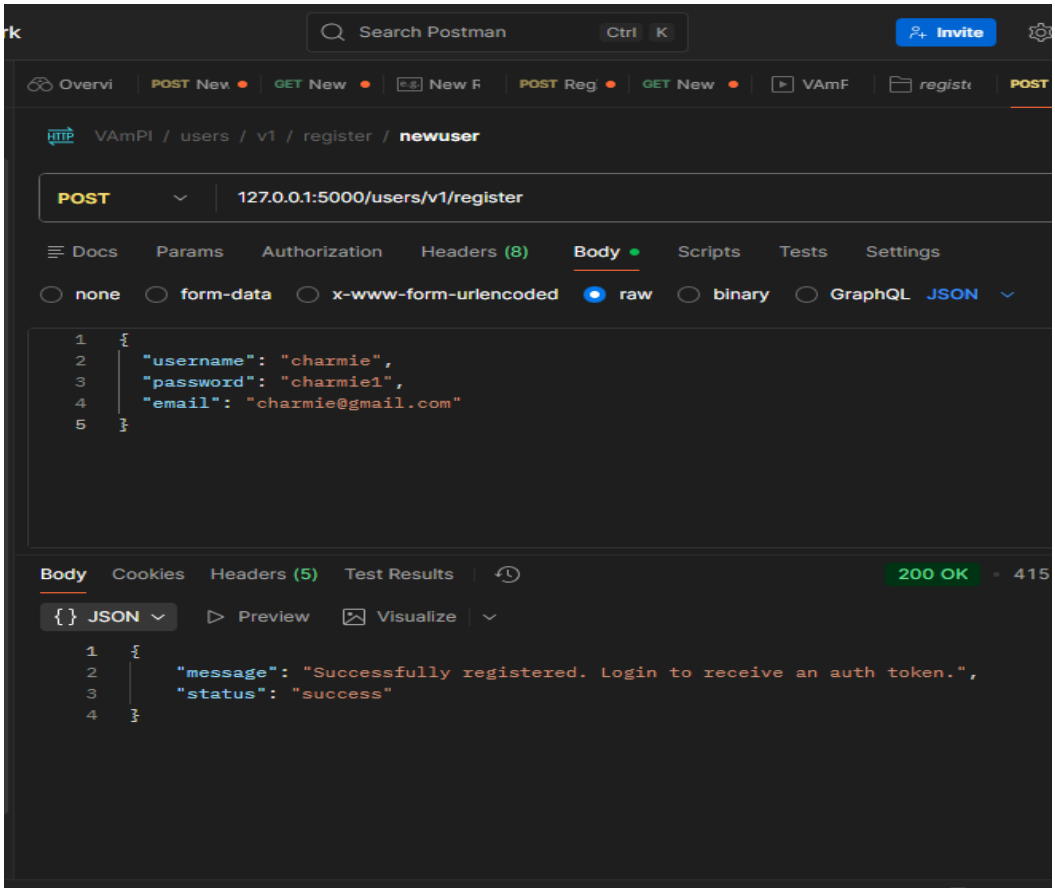
Description:

BOLA is when an API provider does not have sufficient controls in place to enforce authorization. In other words, API users should only have access to sensitive resources that belong to them. When BOLA is present an attacker will be able to access the sensitive data of other users.

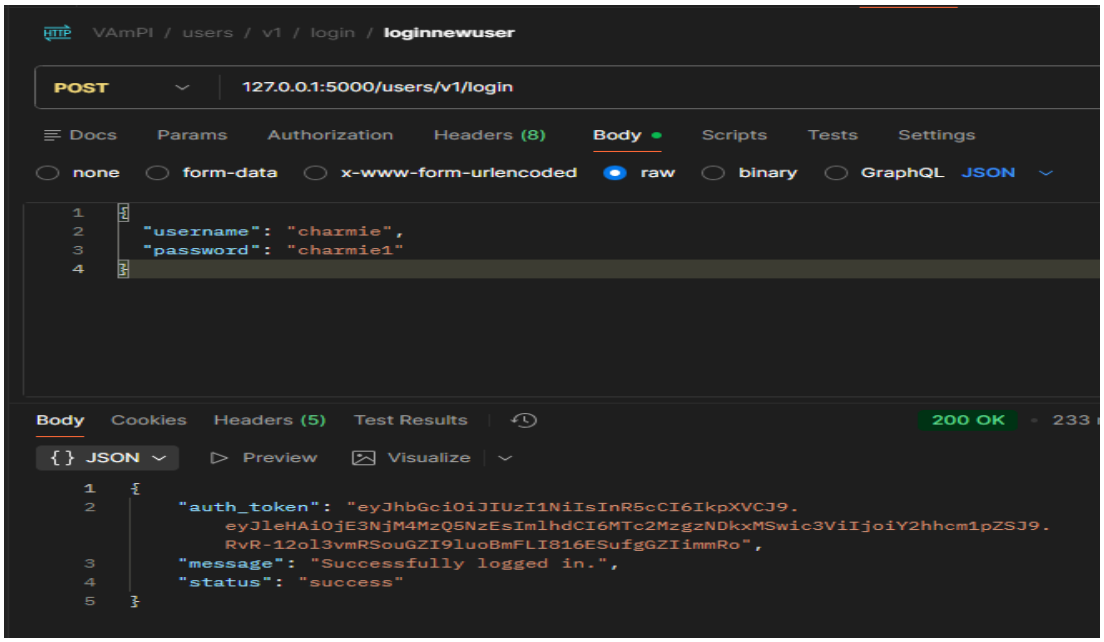
Evidence:

The endpoint **/books/v1/{book_title}** returns sensitive secret information but relies only on basic bearer authentication. The specification claims that only resource owners may retrieve this data, but no ownership check or role validation is documented. This allows authenticated users to access other users' secrets.

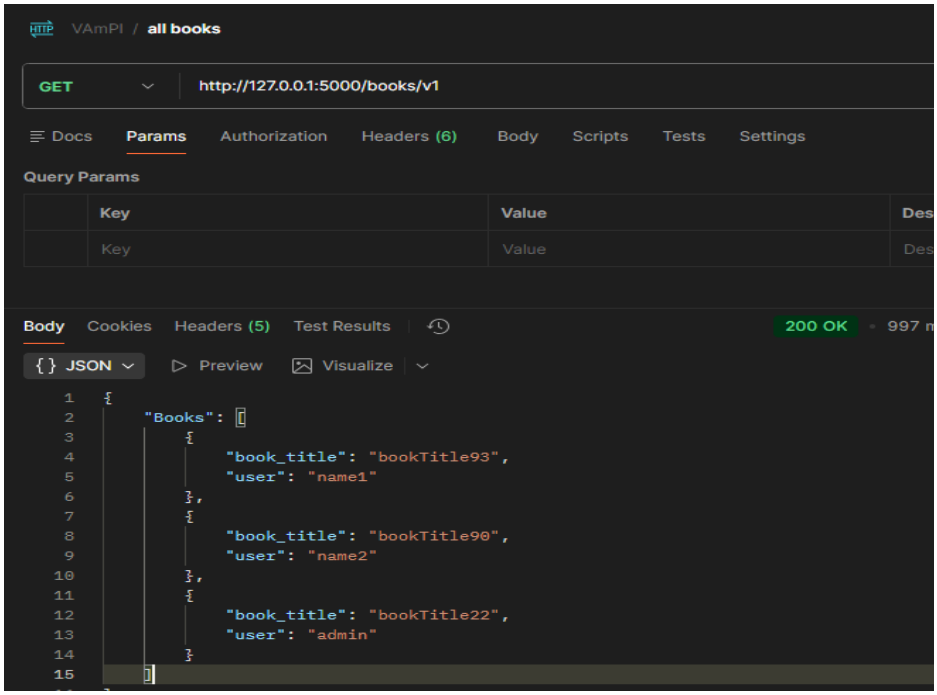
- I created a user (charmie) on the **POST /users/v1/register** endpoint.



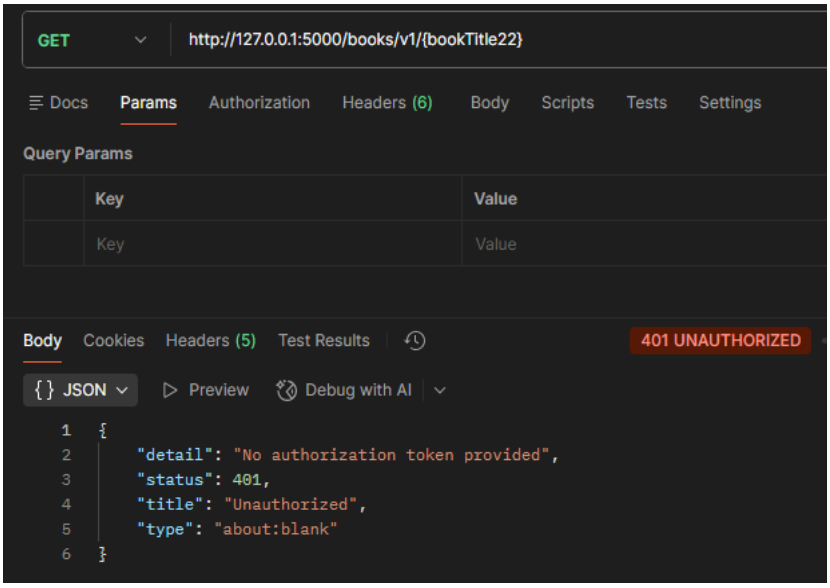
- I successfully logged in as the new user on **POST/users/v1/login** endpoint and got an auth-token in the response.



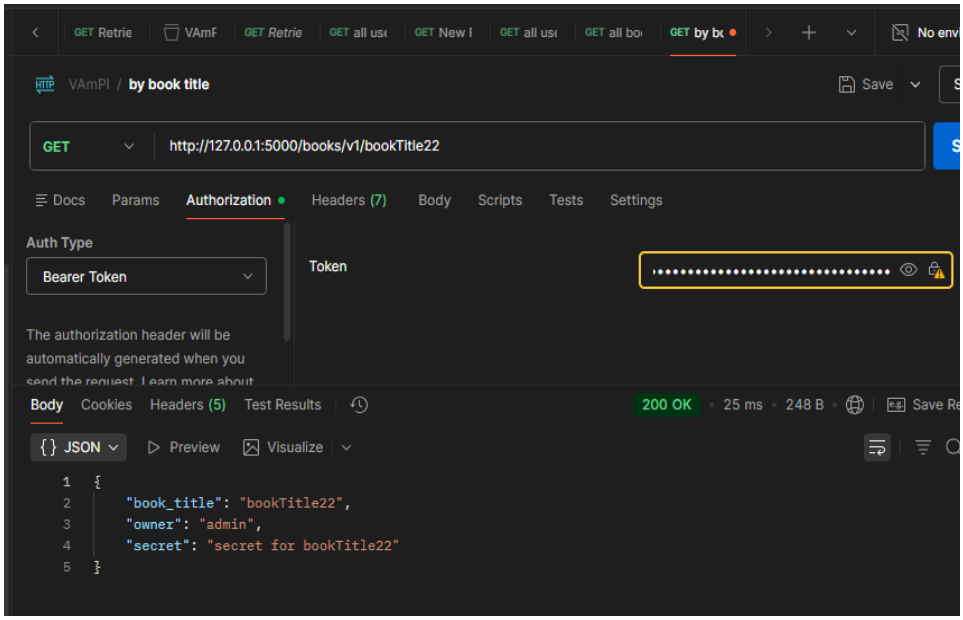
From the end point **/books/v1**, anyone can see the list of all books, even without providing an auth token:



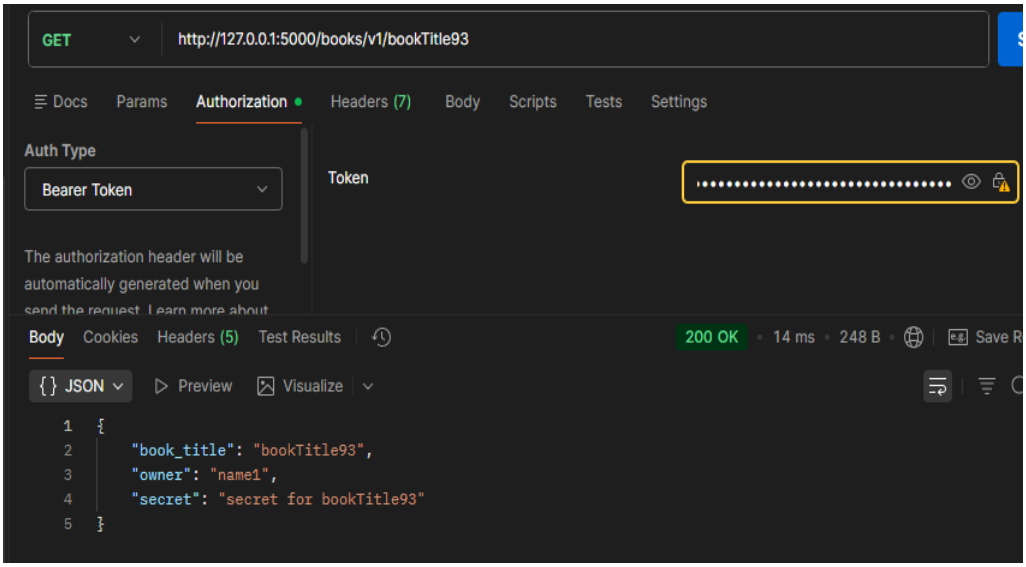
I first tried accessing **/books/v1/bookTitle22** without providing an auth token and it did not bring back the expected secret in the response.



Using the auth token that was sent when the new user logged in as the Bearer Token, I went ahead and accessed another user's (admin) book and their secret via **/books/v1/bookTitle22**, without any authorization checks:any

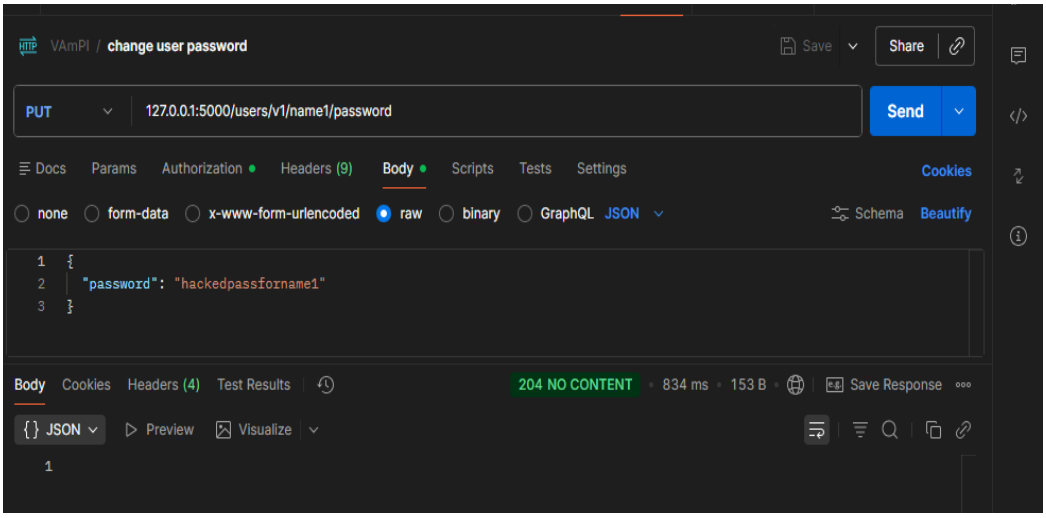


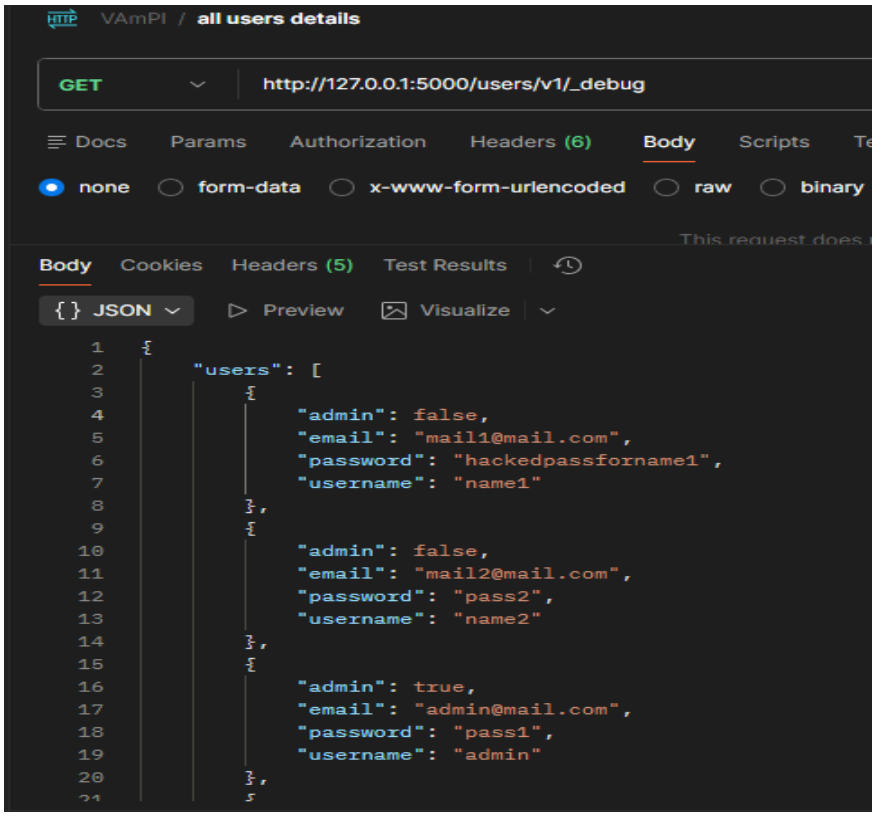
The API shows critical BOLA because the same logged in user is able to access all the other user's books and access their secret. Example **/books/v1/bookTitle93** that belongs to **name1.**



- The endpoint **/users/v1/{username}/password** allows any authenticated user to update the password of any other user simply by changing the **{username}** path parameter. The server does not verify object ownership or enforce role-based authorization. As a result, User A can modify User B's email without permission.

I logged in as the new user that I created and captured the auth token when logging in. I used this token and I was able to change another user's password.





Mitigation:

- Implement server-side ownership checks (**check if the resource owner is the current user**).

- Use role-based access control (RBAC) or OAuth2 scopes.
- Update Swagger to specify required authorization scopes.

2 Excessive Data Exposure - Broken Object Property Level Authorization(BOPLA) OWASP API3

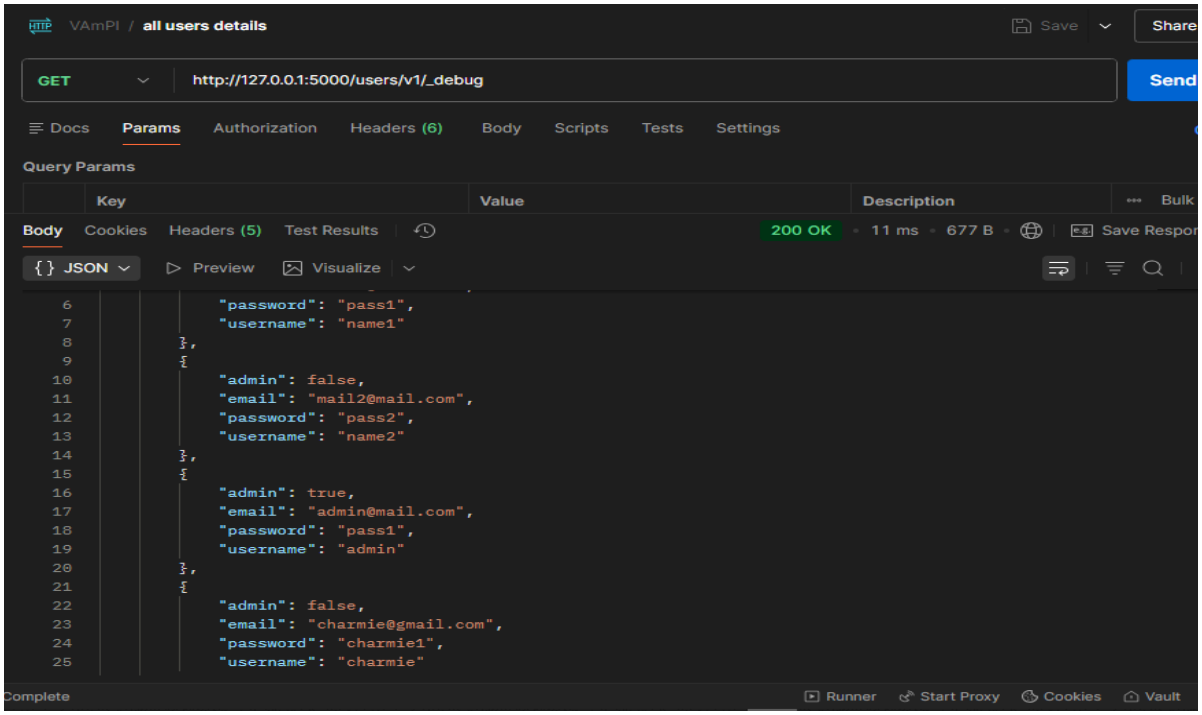
Description:

Excessive Data Exposure takes place when an API provider responds to a request with an entire data object. Usually, an API provider will filter out the data object down to what is being requested. When the data object is shared without being filtered there is an increased risk of exposing sensitive information.

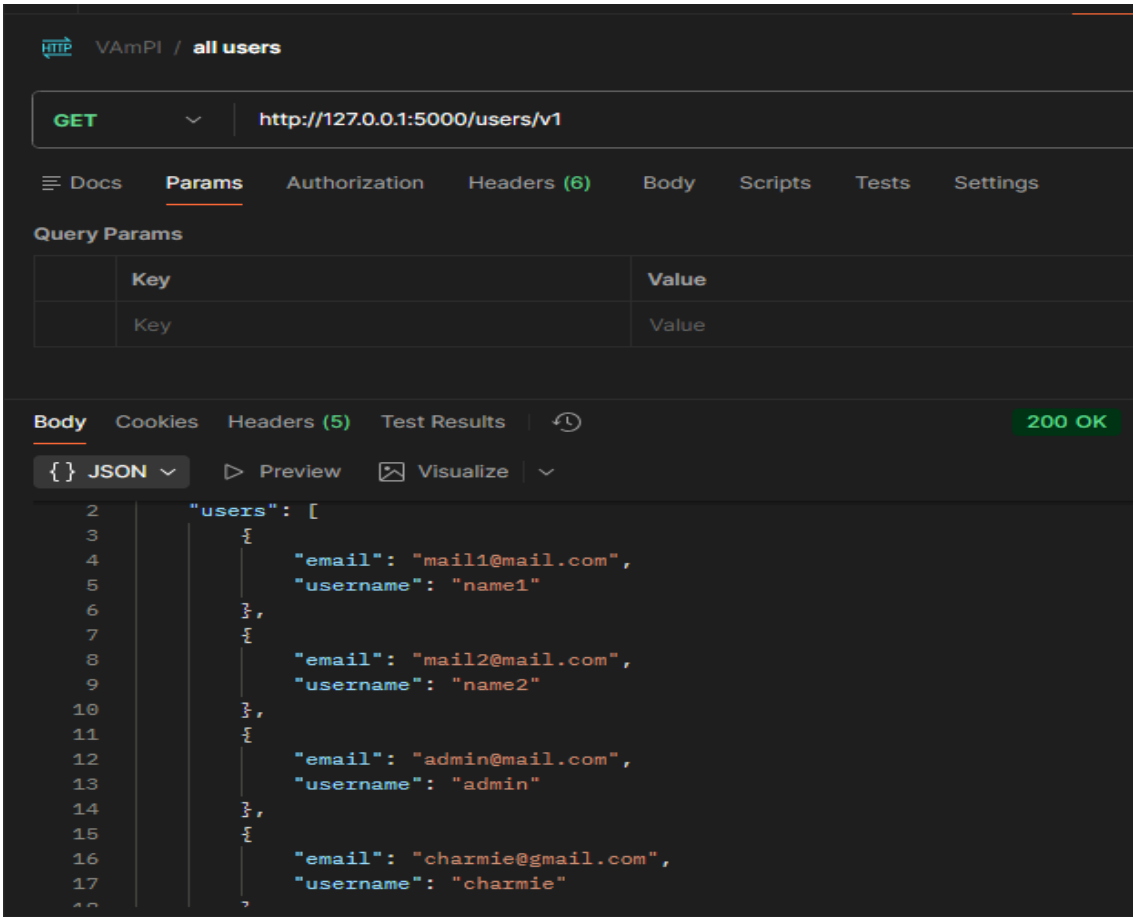
Evidence:

The endpoints **/users/v1/_debug** and **/users/v1** return full user objects including sensitive fields such as password, admin and email and no token was required. This also includes the new user that I just registered. These fields are directly exposed in API responses without filtering. The debug endpoint returns the entire user list. This exposure allows attackers to harvest sensitive data without authentication or with minimal access.

/users/v1/_debug



/users/v1



Mitigation:

- Remove sensitive fields from all API responses.
- Disable or restrict debug endpoints from production.
- Implement response filtering to expose only safe fields.

3 Lack of Access Controls on Sensitive Endpoints – Broken Function Level Authorization OWASP API5

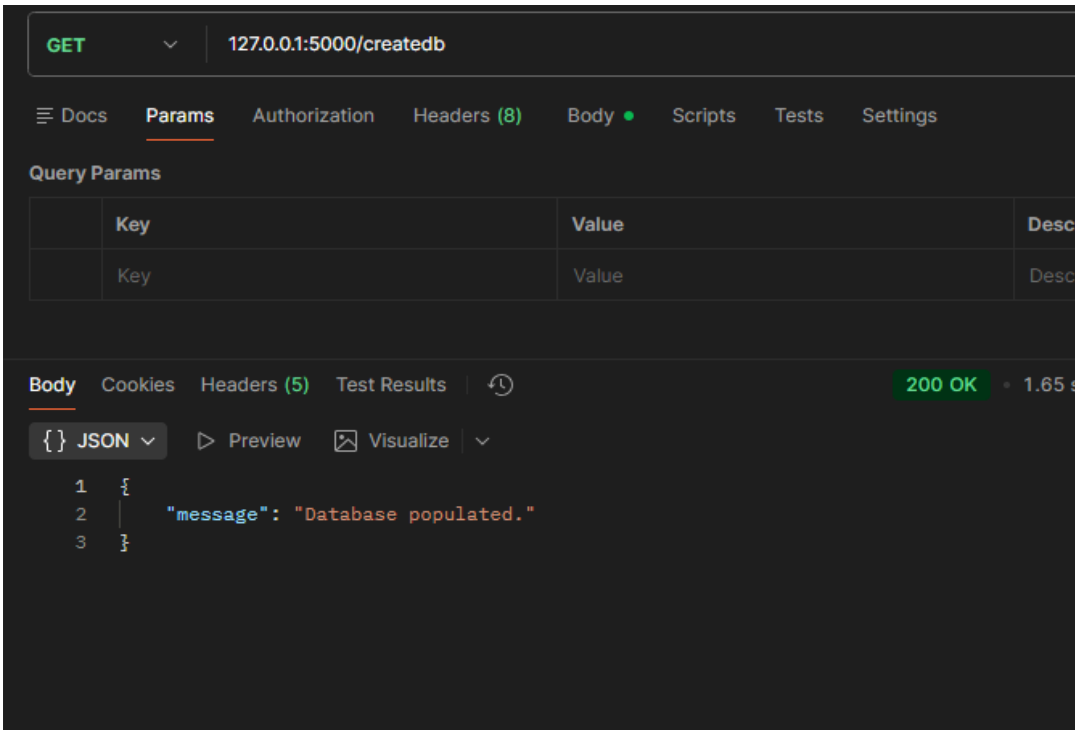
Description

Broken Function Level Authorization is when API functions have insufficient access controls. BFLA is about altering or deleting data. In addition, a vulnerable API would allow an attacker to perform actions of other roles including administrative actions. The **/createdb**, **/users/v1/register and /users/v1/{username}** endpoints are vulnerable to BFLA.

Evidence:

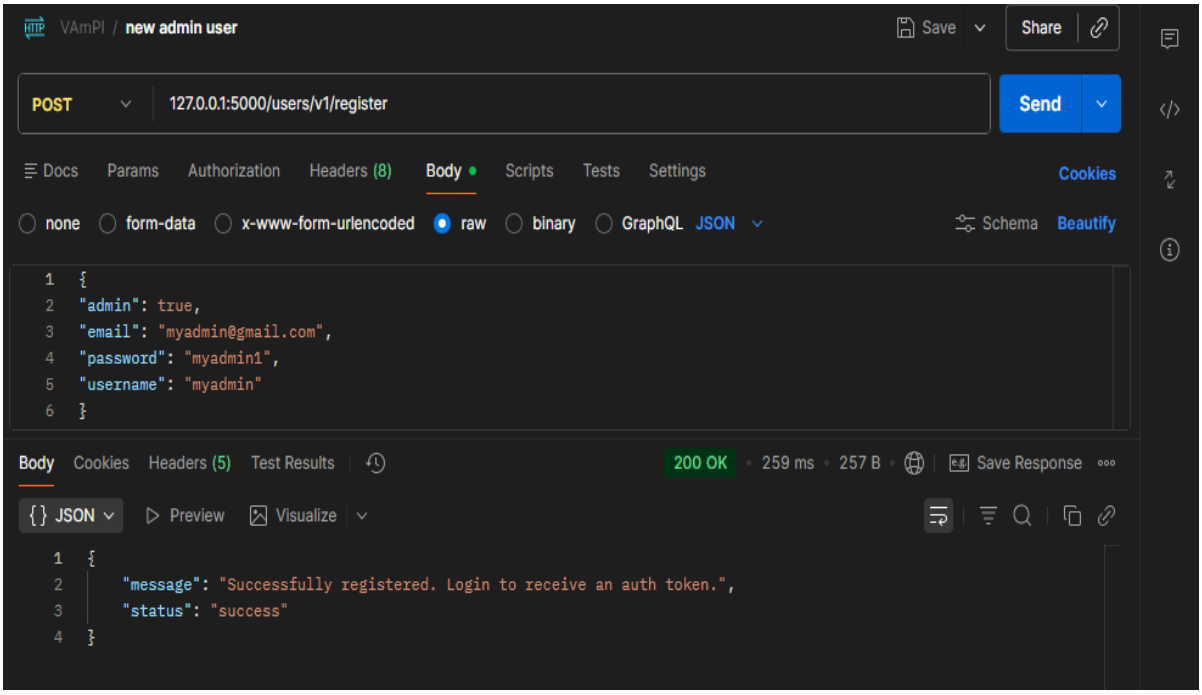
- **Reset Database Without Authentication**

The **/createdb** endpoint allows database creation and populates it without any authentication. As shown in the response message “Database populated.”



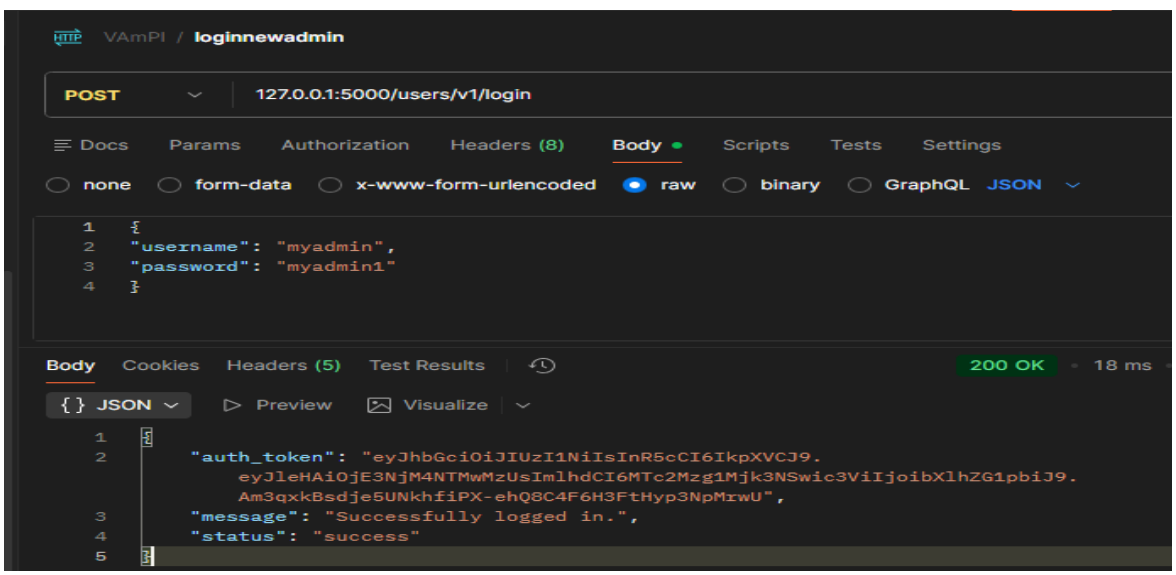
- **Privilege Escalation via User-Controlled Role Assignment**

The signup endpoint **/users/v1/register** allows users to set their own authorization level by including **"admin": true** in the request body. There is no server-side validation preventing normal users from registering as administrators. This results in immediate privilege escalation and full compromise of administrative functionality. This means an attacker is able to use admin functions without any authorization checks, for example deleting users.

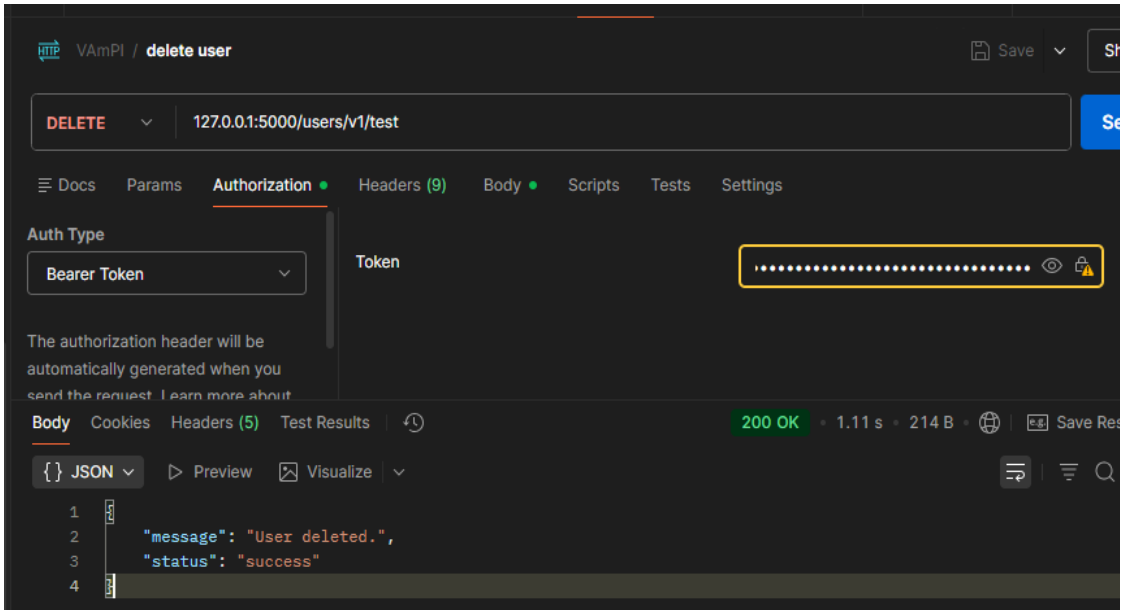


- **Delete Any User**

Now logged in as the new admin(attacker)



Using the admin-only endpoint to delete users(**delete /users/v1/{username}** that relies only on the Bearer Token without enforcing authorization, i managed to delete a test user that i created before. This allows unauthorized destructive operations.



Mitigation:

- Remove development/admin endpoints from production.
- Enforce admin-only role checks on privileged actions like creating admin users and delete.
- Restrict sensitive endpoints via IP whitelisting or environment-based disabling.

Conclusion

The reviewed API specification contains multiple high-risk vulnerabilities. Implementing the recommended mitigations will significantly improve the security posture and protect both user data and backend systems.

End of report

Week 2

Assignment: **JWT Security Analysis Report**
Feedback: **Very good exploit. I like that you tested the tokens on your local server. In future, include recommendations for how to fix vulnerabilities**
Corrections: **Added recommendations for all the vulnerabilities**
Tools used: **Burpsuite Community Edition, xjwt.io**

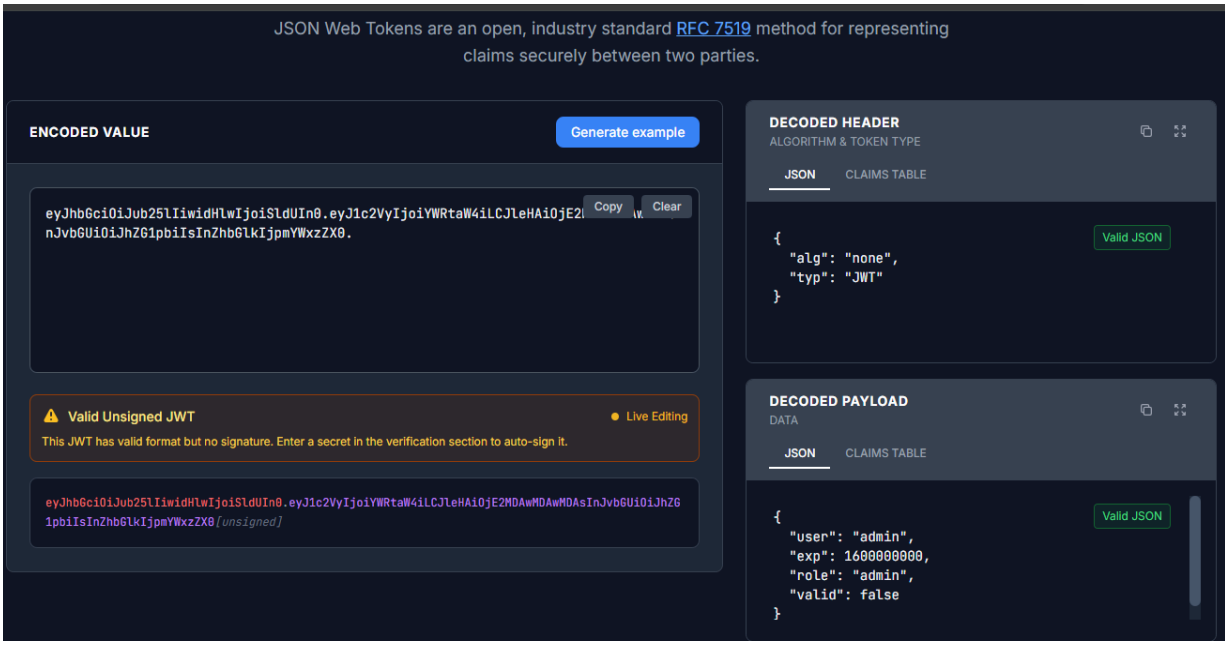
Summary

This report documents the security analysis of a JWT token. The token has multiple critical security vulnerabilities that would allow complete authentication and authorization bypass in any system accepting this token.

5 critical security flaws were identified in the token structure and implementation. These vulnerabilities collectively enable attackers to:

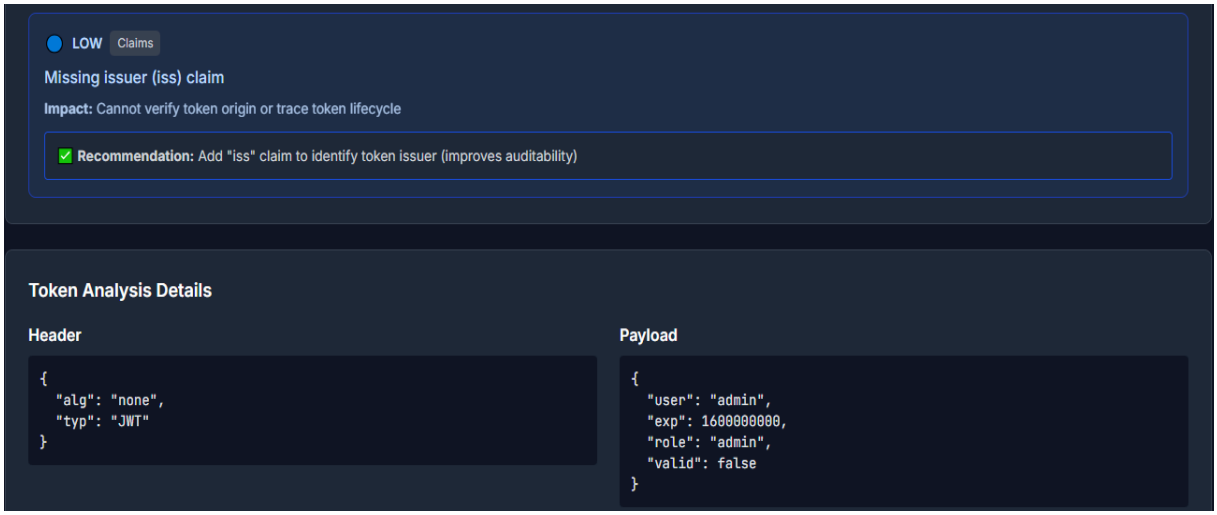
- Forge tokens without cryptographic keys
- Impersonate any user including administrators
- Bypass expiration checks
- Manipulate authorization controls
- Exploit weak validation logic

Critical Finding: This token uses the "none" algorithm meaning that the token is not signed, which is the most severe JWT vulnerability possible and renders all other security controls meaningless. I used xjwt.io's JWT Encoder/Decoder to see the structure of the JWT as shown below:



As shown below, the security flaws are:

- No signature
- Expired token
- Missing timestamp validation(iat,nbf)
- Missing audience claim
- Missing issuer claim

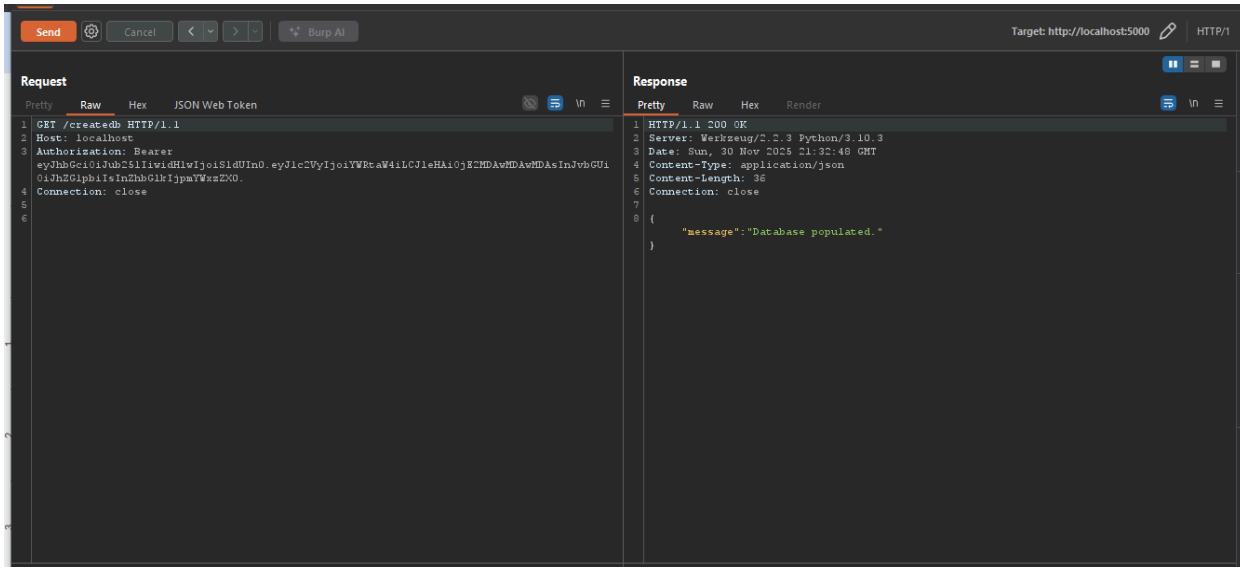


Findings

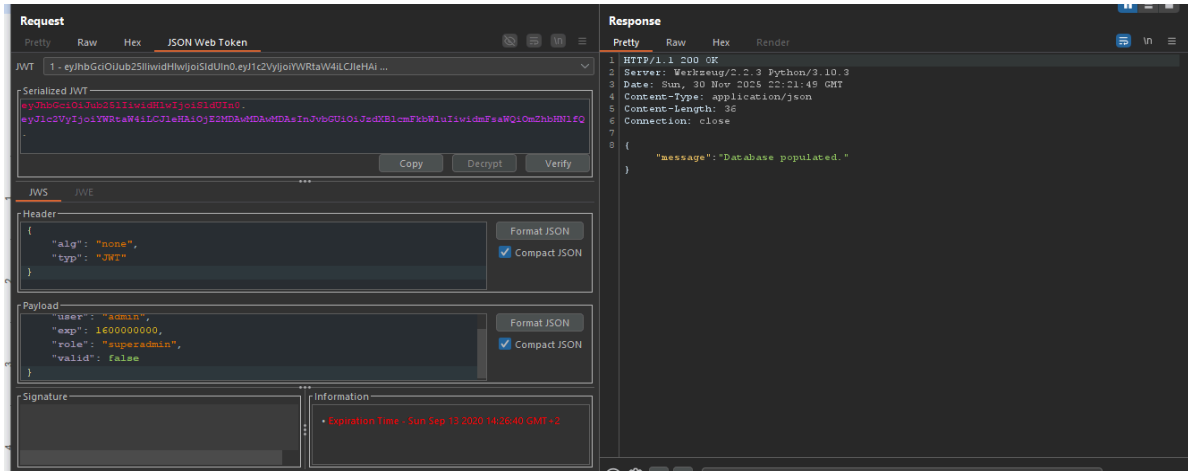
1. No signature algorithm - “alg”:”none”

Why this is a vulnerability: The header contains "alg": "none", meaning the token is unsecured and has no cryptographic signature, so the server cannot detect tampering.

How attackers exploit:
In Burpsuite, I first sent a request to my running backend on localhost:5000 on the /createdb endpoint with the vulnerable JWT in the request header. The server accepted this request.



I modified the role to superadmin, which modifies the payload on the token but the request was still accepted by the server. An attacker can escalate privileges and use this token to access unauthorized data.



Recommendations:

- Completely disable support for the "none" algorithm on both the token issuer and verifier.

- Enforce a strong cryptographic signing algorithm such as:

    - **RS256 (recommended – asymmetric)**

    - or **HS256** (only if secrets are securely managed)

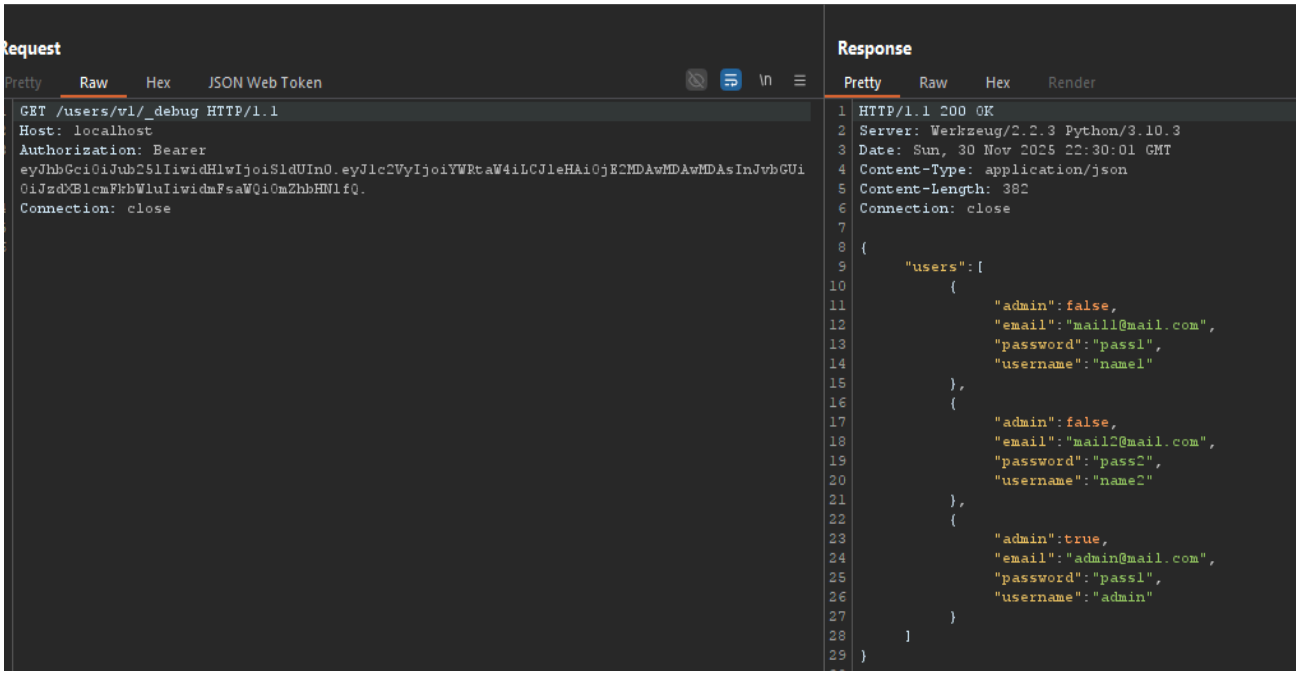
- Ensure the backend explicitly validates the algorithm instead of trusting the value in the JWT header.

2. Token Has Already Expired (exp long in the past)

As shown above, the token expired on Sunday Sep 13 2020 14:26:40 GMT+2.

Why this is a vulnerability:If a server still accepts an expired token, it is vulnerable to token replay attacks, meaning an attacker can reuse old tokens to regain access forever.

How attackers exploit:
I used the same expired token to access a different endpoint and it returned a response. This is exactly how attackers can use this expired token to replay attacks and access APIs forever.



Recommendations:

- Enforce mandatory validation of the exp claim.
- Reject all tokens where:
    - exp is missing
    - exp is in the past
- Configure a reasonable token lifetime (example 15 minutes for access tokens).

3. Missing timestamp validation(iat, nbf)

Why this is a vulnerability: The payload lacks iat (issued at) and nbf (not before) claims, so the backend cannot reliably determine when the token was created or from which time it becomes valid, making it harder to detect abuse, clock skew issues or early/late replays.

How attackers exploit: As shown in the examples above, the token is being accepted without checking freshness and that is how attackers use tokens issued before password resets, logout, or privilege changes.

Recommendations:

- Include and validate:
    - **iat** (Issued At): when the token was created
    - **nbf** (Not Before): earliest time the token is valid
- Apply acceptable clock time(example 2 minutes).

4. Missing audience claim

This means the token can be used across different services.

Why this is a vulnerability: There is no aud (audience) or similar context claim tying the token to a specific service or client, so the same token could be replayed against any system that mistakenly shares the same verification key or trusts the token format

How attackers exploit: An attacker who gets this token from one application could try it against other APIs or environments that accept the same JWTs, achieving cross-service impersonation if any of them do not enforce audience checks.

Recommendations:

- Add an **aud** claim identifying the intended API or service.

The backend must explicitly check that: **aud === expected_service_identifier**

- Use different audience values for different services or environments (example development, staging, production).

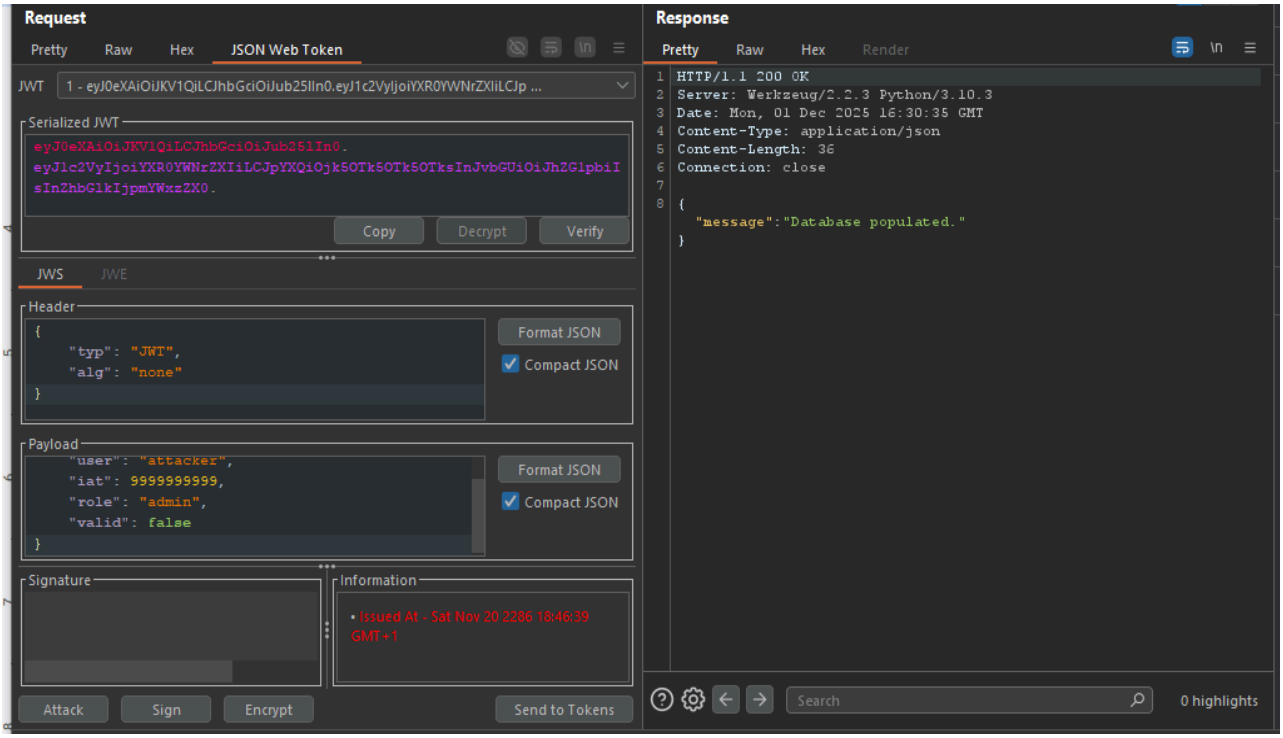
5. Missing issuer claim

Why this is a vulnerability: A secure JWT should include an iss (issuer) claim that identifies which system generated the token. The server cannot verify which system issued the token. Any attacker can generate a fake token from anywhere and the backend cannot detect it. It destroys trust boundaries in microservices / distributed systems and It allows token confusion attacks (sending a JWT from another application to your API)

How attackers exploit:

A missing issuer claim enables the attacker to create their own token and send it to your API, and the API cannot tell if it is fake.

In burpsuit, I modified the payload and set the username to “attacker” but the server accepted it with the role of admin. This means that an attacker can create their own token and access the API.



Recommendations:

- Include an **iss** claim that uniquely identifies the trusted token issuer (e.g., auth server).
- Reject any token where:
  - **iss** is missing
  - **iss** does not match the expected issuer value

Conclusion

The analysis of the provided JWT shows that the JWT is critically insecure due to missing validations, lack of a signature and other absent claims required by the industry standard RFC7519 method. Using Burp Suite, it was demonstrated that an attacker can easily forge or modify the token to gain unauthorized access or escalate privileges. These flaws undermine authentication integrity and expose the application to serious security risks. To secure the system, proper JWT best practices such as enforcing

signed and stronger algorithms(asymmetric), validating claims like issuer and expiration and rejecting tampered tokens must be implemented.

End of report

Final Consolidated Conclusion

This consolidated security report assessed two critical areas of application security: API security controls and JWT-based authentication mechanisms. Across both assessments, multiple high-risk vulnerabilities were identified that could lead to unauthorized access, privilege escalation, data exposure and full system compromise if left unaddressed.

The Week 1 API security assessment of the VAmPI application revealed serious authorization and access control weaknesses, including Broken Object Level Authorization (BOLA), Excessive Data Exposure (BOPLA) and Broken Function Level Authorization (BFLA). These issues demonstrated how attackers could access other users' sensitive data, escalate privileges to administrative roles and perform destructive actions without proper authorization checks. Corrections made after feedback improved the clarity, evidence quality and completeness of the findings, strengthening the overall report.

The Week 2 JWT security analysis further highlighted critical authentication flaws, most notably the use of the "none" algorithm, missing signature validation, acceptance of expired tokens and absence of essential claims such as issuer, audience and timestamps. Practical exploitation using Burp Suite confirmed that attackers can forge tokens, impersonate administrators and bypass all authentication controls. The inclusion of remediation recommendations aligned the findings with industry best practices and standards.

Overall, this report demonstrates that weak authorization logic and improper token validation pose severe risks to application security. Implementing strong access controls, enforcing least privilege, validating ownership and roles and following secure JWT best practices are essential to protecting APIs and backend systems. Addressing these vulnerabilities will significantly improve the security posture of the application and reduce the likelihood of real-world exploitation.

End of Consolidated Security Report